

Nama : Edsel Sulthan Farrel

NIM : 152024166

LAPORAN PRAKTIKUM

Arsitektur Paralel: SISD, SIMD, MISD, dan MIMD

1. Pendahuluan

Dalam komputasi modern, terdapat beberapa model arsitektur pemrosesan data yang diklasifikasikan berdasarkan cara instruksi dan data diproses, yaitu:

1. SISD (Single Instruction Single Data)
2. SIMD (Single Instruction Multiple Data)
3. MISD (Multiple Instruction Single Data)
4. MIMD (Multiple Instruction Multiple Data)

Praktikum ini bertujuan untuk memahami perbedaan keempat model tersebut melalui implementasi program Python.

2. Pembahasan Program

2.1 SISD (Single Instruction Single Data)

Kode Program

SISD: proses satu data per satu waktu

```
numbers = [10, 20, 30, 40, 50]
```

```
result = []
```

```
for num in numbers:
```

```
    result.append(num * 2) # Satu instruksi, satu data
```

```
print(result) # [20, 40, 60, 80, 100]
```

Output:

```
[20, 40, 60, 80, 100]
```

Analisis:

- Instruksi $\text{num} * 2$ dieksekusi satu per satu secara berurutan.
- Tidak ada paralelisme — setiap iterasi menunggu iterasi sebelumnya selesai.
- Cocok untuk tugas yang memiliki ketergantungan antar data (data dependency).
- Waktu eksekusi berbanding lurus dengan jumlah elemen data ($O(n)$).

2.2 SIMD (Single Instruction Multiple Data)

Kode Program

```
import numpy as np

# NumPy menggunakan SIMD di belakang layar secara otomatis
a = np.array([1, 2, 3, 4, 5, 6, 7, 8], dtype=np.float32)
b = np.array([10, 20, 30, 40, 50, 60, 70, 80], dtype=np.float32)

# Satu operasi → semua elemen diproses paralel (SIMD internally)
result = a + b

print(result)
# Output: [11. 22. 33. 44. 55. 66. 77. 88.]
```

Output:

```
[11. 22. 33. 44. 55. 66. 77. 88.]
```

Analisis:

- Operasi $a + b$ menerapkan instruksi penjumlahan yang sama ke semua 8 pasang elemen secara simultan.
- NumPy secara internal memanfaatkan register SIMD (128-bit atau 256-bit) pada CPU modern.
- Performa jauh lebih tinggi dibanding SISD untuk operasi vektor/matriks berskala besar.
- Sangat relevan pada aplikasi deep learning, image processing, dan scientific computing.

2.3 MISD (Multiple Instruction Single Data)

Kode Program

```
from multiprocessing import Process

# SATU DATA yang sama
data = 100
```

```

def instruksi_A(x):
    hasil = x + 50
    print(f"[Instruksi A] {x} + 50 = {hasil}")

def instruksi_B(x):
    hasil = x * 2
    print(f"[Instruksi B] {x} * 2 = {hasil}")

def instruksi_C(x):
    hasil = x ** 2
    print(f"[Instruksi C] {x} ** 2 = {hasil}")

def instruksi_D(x):
    hasil = x - 30
    print(f"[Instruksi D] {x} - 30 = {hasil}")

if __name__ == "__main__":
    # Banyak instruksi BERBEDA → data SAMA (100)
    proses = [
        Process(target=instruksi_A, args=(data,)),
        Process(target=instruksi_B, args=(data,)),
        Process(target=instruksi_C, args=(data,)),
        Process(target=instruksi_D, args=(data,)),
    ]

    for p in proses:
        p.start()

    for p in proses:
        p.join()

# ```

```

```
# **Output:**  
  
# ```  
  
# [Instruksi A] 100 + 50 = 150  
# [Instruksi B] 100 * 2 = 200  
# [Instruksi C] 100 ** 2 = 10000  
# [Instruksi D] 100 - 30 = 70
```

Output:

```
[Instruksi A] 100 + 50 = 150  
[Instruksi B] 100 * 2 = 200  
[Instruksi C] 100 ** 2 = 10000  
[Instruksi D] 100 - 30 = 70
```

Analisis:

- Satu data (nilai 100) dikirim ke empat proses berbeda secara bersamaan.
- Masing-masing proses menerapkan instruksi yang berbeda (penjumlahan, perkalian, pangkat, pengurangan).
- Keempat proses berjalan paralel menggunakan modul multiprocessing Python.
- Penggunaan nyata MISD: sistem avionik dengan triple modular redundancy (TMR) untuk fault detection.

2.4 MIMD (Multiple Instruction Multiple Data)

Kode Program

```
from multiprocessing import Process, current_process  
  
import os  
  
def tugas_A(data):  
  
    result = [x * 3 for x in data]  
    print(f"[{current_process().name}] Multiply: {result}")  
  
def tugas_B(data):  
  
    result = [x for x in data if x % 2 == 0]
```

```

print(f"[{current_process().name}] Filter: {result}")

def tugas_C(data):

    result = sum(data)
    print(f"[{current_process().name}] Sum: {result}")

if __name__ == "__main__":
    data1 = [1, 2, 3, 4]
    data2 = [5, 6, 7, 8]
    data3 = [9, 10, 11, 12]

    # 3 proses, instruksi BERBEDA, data BERBEDA → MIMD
    p1 = Process(target=tugas_A, args=(data1,), name="Process-A")
    p2 = Process(target=tugas_B, args=(data2,), name="Process-B")
    p3 = Process(target=tugas_C, args=(data3,), name="Process-C")

    p1.start(); p2.start(); p3.start()
    p1.join(); p2.join(); p3.join()

# **Output:**
# ```
# [Process-A] Multiply: [3, 6, 9, 12]
# [Process-B] Filter: [6, 8]
# [Process-C] Sum: 42
# ```

```

Output:

```

[Process-A] Multiply: [3, 6, 9, 12]
[Process-B] Filter: [6, 8]
[Process-C] Sum: 42

```

Analisis:

- Process-A melakukan operasi perkalian pada data1 = [1, 2, 3, 4].
- Process-B melakukan filter bilangan genap pada data2 = [5, 6, 7, 8], menghasilkan [6, 8].
- Process-C menghitung jumlah total data3 = [9, 10, 11, 12], menghasilkan 42.
- Ketiga proses berjalan paralel dengan memori terpisah (independent memory space).
- MIMD adalah dasar dari multi-core CPU modern, Hadoop, dan sistem komputasi terdistribusi.